# CSE4095 Introduction to NLP

Assoc. Prof. Dr. Murat Can Ganiz
Marmara University
Collage of Engineering
Computer Engineering
@BIGDaTA_Lab

Slides credit: Dan Jurafsky and mainly from Marine Carpuat

## Lecture 2: Vector Representations of Texts

- Reminders
  - There will be ZOOM polls every week for attendance. If you miss more than 30% of these polls you may get DZ (fail from attendance)
  - I may open the poll any time in the lecture as you have to follow the lecture from beginning to end. These will be %10 of your grade.
  - There will be homeworks during the semester, mainly readings from your text books. There will be pop quizes from these homeworks during the semester. These will be %10 of your grade either.
  - First homework: Read Chapter 6 of Speech and Language Processing. Daniel Jurafsky & James H. Martin. Copyright © 2020. There may be a quiz related to this chapter on Wednesday or following weeks.

2

# Reminders: Text Books

- Speech and Language Processing. Daniel Jurafsky & James H. Martin. 2020.
  - https://web.stanford.edu/~jurafsky/slp3/
- Linguistic Fundamentals for Natural Language Processing: 100 Essentials from Morphology and Syntax (Synthesis Lectures on Human Language Technologies), June 2013, 184 pages, Emily M. Bender, University of Washington
  - https://doi.org/10.2200/S00493ED1V01Y201303HLT020
- Natural Language Processing by Jacob Eisenstein, 2018
  - https://github.com/jacobeisenstein/gt-nlp-class/blob/master/notes/eisenstein-nlp-notes.pdf
- Foundations of Statistical Natural Language Processing by Chris Manning and Hinrich Schuetze, 199
  - https://nlp.stanford.edu/fsnlp/
- A Course in Machine Learning by Hal Daumé III
  - http://ciml.info/

3

# Reminders: Grading

- Attendance %10
- Quizes %10
- Midterm Exam %20
- Final Exam %40
- Project %20

(Subject to change)

4

## Words & their Meaning

- 2 core issues from an NLP perspective
- **Semantic similarity**: given two words, how similar are they in meaning?
- **Word sense disambiguation**: given a word that has more than one meaning, which one is used in a specific context?

5

## Word similarity for question answering

- "**fast**" is similar to "**rapid**" "**tall**" is similar to "**height**"
- Question answering:
- *Q: "How **tall** is Mt. Everest?"*
  *Candidate A: "The official **height** of Mount Everest is 8849 m"*

6

# Vector Space Model

- Term x document matrix
- vector space model of information retrieval (Salton, 1971)
- Information retrieval (IR) is the task of finding the document *d* from the *D* documents in some collection that best matches a query *q*

| | As You Like It | Twelfth Night | Julius Caesar | Henry V |
|---|---|---|---|---|
| **battle** | 1 | 0 | 7 | 13 |
| **good** | 114 | 80 | 62 | 89 |
| **fool** | 36 | 58 | 1 | 4 |
| **wit** | 20 | 15 | 2 | 3 |

**Figure 6.2** The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

7

# Vector Space Model

- Term x document matrix
- vector space model of information retrieval (Salton, 1971)

| | As You Like It | Twelfth Night | Julius Caesar | Henry V |
|---|---|---|---|---|
| **battle** | 1 | 0 | 7 | 13 |
| **good** | 114 | 80 | 62 | 89 |
| **fool** | 36 | 58 | 1 | 4 |
| **wit** | 20 | 15 | 2 | 3 |

**Figure 6.3** The term-document matrix for four words in four Shakespeare plays. The red boxes show that each document is represented as a column vector of length four.

8

# Vector Space Model

- Term x document matrix
- vector space model of information retrieval (Salton, 1971)

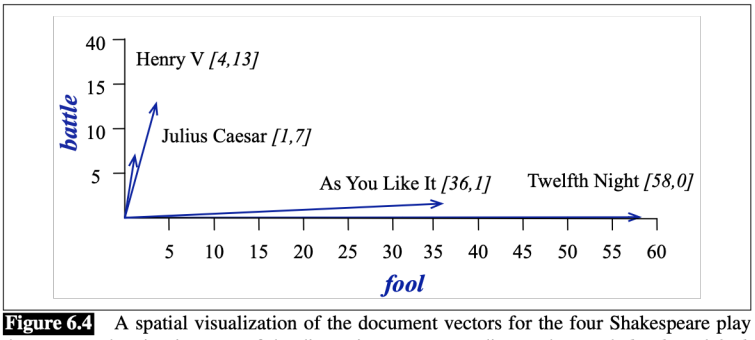



**Figure 6.4** A spatial visualization of the document vectors for the four Shakespeare play documents, showing just two of the dimensions, corresponding to the words *battle* and *fool*. The comedies have high values for the *fool* dimension and low values for the *battle* dimension.

9

# Words as vectors: document dimensions

| | As You Like It | Twelfth Night | Julius Caesar | Henry V |
|---|---|---|---|---|
| **battle** | 1 | 0 | 7 | 13 |
| **good** | 114 | 80 | 62 | 89 |
| **fool** | 36 | 58 | 1 | 4 |
| **wit** | 20 | 15 | 2 | 3 |

**Figure 6.5** The term-document matrix for four words in four Shakespeare plays. The red boxes show that each word is represented as a row vector of length four.

10

# Words as vectors: word dimensions

- An alternative to using the term-document matrix to represent words as vectors of document counts, is to use the **term-term matrix**, also called the **word-word matrix** or the **term-context matrix**

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

If we then take every occurrence of each word (say **strawberry**) and count the context words around it, we get a word-word co-occurrence matrix. Fig. 6.6 shows a simplified subset of the word-word co-occurrence matrix for these four words computed from the Wikipedia corpus (Davies, 2015).

| | aardvark | ... | computer | data | result | pie | sugar | ... |
|---|---|---|---|---|---|---|---|---|
| **cherry** | 0 | ... | 2 | 8 | 9 | 442 | 25 | ... |
| **strawberry** | 0 | ... | 0 | 0 | 1 | 60 | 19 | ... |
| **digital** | 0 | ... | 1670 | 1683 | 85 | 5 | 4 | ... |
| **information** | 0 | ... | 3325 | 3982 | 378 | 5 | 13 | ... |

**Figure 6.6** Co-occurrence vectors for four words in the Wikipedia corpus, showing six of the dimensions (hand-picked for pedagogical purposes). The vector for *digital* is outlined in red. Note that a real vector would have vastly more dimensions and thus be much sparser.

11

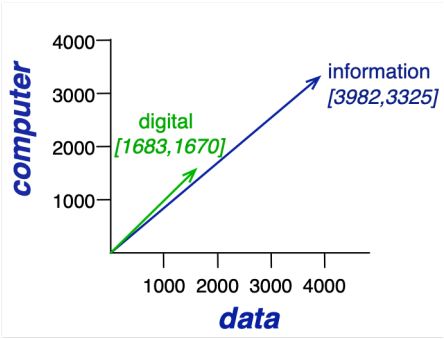
# Words as vectors: word dimensions

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

If we then take every occurrence of each word (say **strawberry**) and count the context words around it, we get a word-word co-occurrence matrix. Fig. 6.6 shows a simplified subset of the word-word co-occurrence matrix for these four words computed from the Wikipedia corpus (Davies, 2015).

| | aardvark | ... | computer | data | result | pie | sugar | ... |
|---|---|---|---|---|---|---|---|---|
| **cherry** | 0 | ... | 2 | 8 | 9 | 442 | 25 | ... |
| **strawberry** | 0 | ... | 0 | 0 | 1 | 60 | 19 | ... |
| **digital** | 0 | ... | 1670 | 1683 | 85 | 5 | 4 | ... |
| **information** | 0 | ... | 3325 | 3982 | 378 | 5 | 13 | ... |

**Figure 6.6** Co-occurrence vectors for four words in the Wikipedia corpus, showing six of the dimensions (hand-picked for pedagogical purposes). The vector for *digital* is outlined in red. Note that a real vector would have vastly more dimensions and thus be much sparser.




12

# Cosine for measuring similarity

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^{N} v_i^2}$$

$$\text{dot product}(\mathbf{v},\mathbf{w}) = \mathbf{v}\cdot\mathbf{w} = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + v_2 w_2 + \ldots + v_N w_N$$

$$\mathbf{a}\cdot\mathbf{b} = |\mathbf{a}||\mathbf{b}|\cos\theta$$

$$\frac{\mathbf{a}\cdot\mathbf{b}}{|\mathbf{a}||\mathbf{b}|} = \cos\theta$$

$$\text{cosine}(\mathbf{v},\mathbf{w}) = \frac{\mathbf{v}\cdot\mathbf{w}}{|\mathbf{v}||\mathbf{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

13

# Cosine for measuring similarity

$$\text{cosine}(\mathbf{v},\mathbf{w}) = \frac{\mathbf{v}\cdot\mathbf{w}}{|\mathbf{v}||\mathbf{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

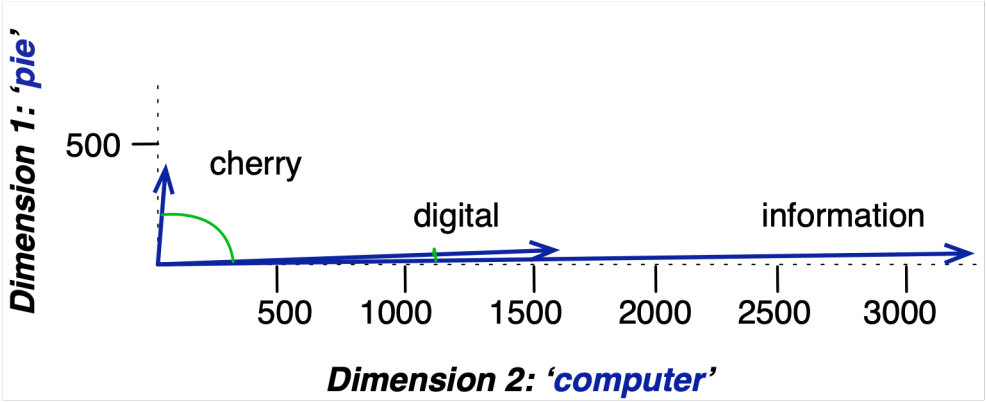
| | pie | data | computer |
|---|---|---|---|
| **cherry** | 442 | 8 | 2 |
| **digital** | 5 | 1683 | 1670 |
| **information** | 5 | 3982 | 3325 |

$$\cos(\text{cherry},\text{information}) = \frac{442*5+8*3982+2*3325}{\sqrt{442^2+8^2+2^2}\sqrt{5^2+3982^2+3325^2}} = .017$$

$$\cos(\text{digital},\text{information}) = \frac{5*5+1683*3982+1670*3325}{\sqrt{5^2+1683^2+1670^2}\sqrt{5^2+3982^2+3325^2}} = .996$$

14

# Cosine Similarity




15

# Term Weighting

- Binary {0,1}
- Term frequency
  - $tf(t_i,d_j) = count(t_i,d_j)$
  - $tf(t_i,d_j) = \log_{10}(count(t_i,d_j)+1)$
- Document frequency
  - $df(t_i)$ = # of documents that includes $t_i$
- Inverse Document Frequency $\quad \mathrm{idf}_t = \log_{10}\left(\frac{N}{\mathrm{df}_t}\right)$
- **tf-idf**
  - $Tf\text{-}idf(t_i,d_j) = \log_{10}(count(t_i,d_j)+1)\ \log_{10}(N/count(t_i))$

16

## tf-idf

| | As You Like It | Twelfth Night | Julius Caesar | Henry V |
|---|---|---|---|---|
| **battle** | 0.074 | 0 | 0.22 | 0.28 |
| **good** | 0 | 0 | 0 | 0 |
| **fool** | 0.019 | 0.021 | 0.0036 | 0.0083 |
| **wit** | 0.049 | 0.044 | 0.018 | 0.022 |

**Figure 6.9** A tf-idf weighted term-document matrix for four words in four Shakespeare plays, using the counts in Fig. 6.2. For example the 0.049 value for *wit* in *As You Like It* is the product of $\text{tf} = \log_{10}(20+1) = 1.322$ and $\text{idf} = .037$. Note that the idf weighting has eliminated the importance of the ubiquitous word *good* and vastly reduced the impact of the almost-ubiquitous word *fool*.

17

## Pointwise Mutual Information (PMI)

$$I(x,y) = \log_2 \frac{P(x,y)}{P(x)P(y)} \qquad I(X,Y) = \sum_x \sum_y P(x,y) \log_2 \frac{P(x,y)}{P(x)P(y)}$$

The pointwise mutual information between a target word $w$ and a context word $c$ (Church and Hanks 1989, Church and Hanks 1990) is then defined as:

$$\text{PMI}(w,c) = \log_2 \frac{P(w,c)}{P(w)P(c)} \tag{6.17}$$

$$\text{PPMI}(w,c) = \max(\log_2 \frac{P(w,c)}{P(w)P(c)}, 0)$$

18

# Pointwise Mutual Information (PMI)

$f_{ij}$ gives the number of times word $w_i$ occurs in context $c_j$. This can be turned into a PPMI matrix where $ppmi_{ij}$ gives the PPMI value of word $w_i$ with context $c_j$ as follows:

$$p_{ij} = \frac{f_{ij}}{\sum_{i=1}^{W}\sum_{j=1}^{C} f_{ij}} \quad p_{i*} = \frac{\sum_{j=1}^{C} f_{ij}}{\sum_{i=1}^{W}\sum_{j=1}^{C} f_{ij}} \quad p_{*j} = \frac{\sum_{i=1}^{W} f_{ij}}{\sum_{i=1}^{W}\sum_{j=1}^{C} f_{ij}}$$

$$\text{PPMI}_{ij} = \max(\log_2 \frac{p_{ij}}{p_{i*}p_{*j}}, 0)$$

19

# Pointwise Mutual Information (PMI)

| | computer | data | result | pie | sugar | count(w) |
|---|---|---|---|---|---|---|
| **cherry** | 2 | 8 | 9 | 442 | 25 | 486 |
| **strawberry** | 0 | 0 | 1 | 60 | 19 | 80 |
| **digital** | 1670 | 1683 | 85 | 5 | 4 | 3447 |
| **information** | 3325 | 3982 | 378 | 5 | 13 | 7703 |
| **count(context)** | 4997 | 5673 | 473 | 512 | 61 | 11716 |

**Figure 6.10** Co-occurrence counts for four words in 5 contexts in the Wikipedia corpus, together with the marginals, pretending for the purpose of this calculation that no other words/contexts matter.

$$P(\text{w=information,c=data}) = \frac{3982}{11716} = .3399$$
$$P(\text{w=information}) = \frac{7703}{11716} = .6575$$
$$P(\text{c=data}) = \frac{5673}{11716} = .4842$$
$$\text{ppmi(information,data)} = \log 2(.3399/(.6575 * .4842)) = .0944$$

20

# Pointwise Mutual Information (PMI)

$$\text{PPMI}(w,c) = \max(\log_2 \frac{P(w,c)}{P(w)P(c)}, 0)$$

$$\text{PPMI}_\alpha(w,c) = \max(\log_2 \frac{P(w,c)}{P(w)P_\alpha(c)}, 0)$$

$$P_\alpha(c) = \frac{count(c)^\alpha}{\sum_c count(c)^\alpha}$$

| | computer | data | result | pie | sugar |
|---|---|---|---|---|---|
| **cherry** | 0 | 0 | 0 | 4.38 | 3.30 |
| **strawberry** | 0 | 0 | 0 | 4.10 | 5.51 |
| **digital** | 0.18 | 0.01 | 0 | 0 | 0 |
| **information** | 0.02 | 0.09 | 0.28 | 0 | 0 |

**Figure 6.12** The PPMI matrix showing the association between words and context words, computed from the counts in Fig. 6.11. Note that most of the 0 PPMI values are ones that had a negative PMI; for example PMI(*cherry,computer*) = -6.7, meaning that *cherry* and *computer* co-occur on Wikipedia less often than we would expect by chance, and with PPMI we replace negative values by zero.

21

# Word2vec

- More powerful word representation: embeddings, short dense vectors.
- Vector size -> 50-1000
- Vector size for tf-idf -> |D| or |V|  10k-100k
- word2vec (Mikolov et al. 2013, Mikolov et al. 2013a)
- skip-gram with negative sampling, sometimes called SGNS
- static embeddings, meaning that the method learns one fixed embedding for each word in the vocabulary
  - contextual em- beddings like the popular BERT or ELMO representations, in which the vector for each word is different in different contexts.

22

# Word2Vec

- Instead of counting how often each word *w* oc- curs near, say, *apricot*, we'll instead train a classifier on a binary prediction task: "Is word *w* likely to show up near *apricot*?"
- We don't care about the prediction, instead we'll take the learned classifier *weights* as the word embeddings
- **Self-supervision**: use running text as implicitly supervised training data for such a classifier; a word *c* that occurs near the target word *apricot* acts as gold 'correct answer' to "Is word *c* likely to show up near *apricot*?"
- training a logistic regression classifier instead of a multi-layer neural network with hidden layers that demand more sophisticated training algorithms

23

# Other static embeddings

- fasttext (Bojanowski et al., 2017)
  - deals with unknown words and sparsity in languages with rich morphology, by using subword models. Each word in fasttext is represented as itself plus a bag of constituent n-grams
  - with *n* = 3 the word *where* would be represented by the sequence <where> plus the character n-grams: <wh, whe, her, ere, re>
  - Skip-gram for each n-gram
- GloVe (Pennington et al., 2014),
  - GloVe is based on ratios of probabilities from the word-word co-occurrence matrix, combining the intuitions of count-based models like PPMI while also capturing the linear structures used by methods like word2vec

24

# Semantic properties of embeddings

- first-order co-occurrence
  - Two words have first-order co-occurrence (sometimes called syntagmatic association) if they are typically nearby each other. Thus *wrote* is a first-order associate of *book* or *poem*.
- second-order co-occurrence
  - Two words have second-order co-occurrence (sometimes called paradigmatic association) if they have similar neighbors. Thus *wrote* is a second-order associate of words like *said* or *remarked*.

25

# Semantic properties of embeddings

- Analogy/Relational Similarity
  - Another semantic property of embeddings is their ability to capture relational meanings. In an important early vector space model of cognition, Rumelhart and Abrahamson (1973) proposed the parallelogram model for solving simple analogy problems of the form *a is to b as a* is to what?*

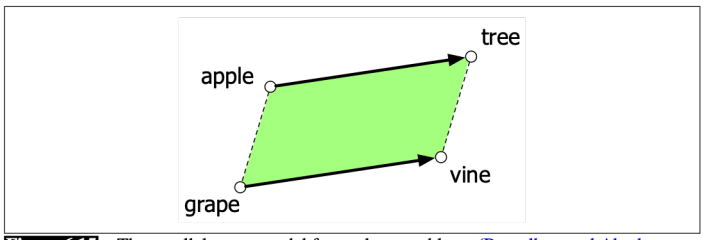



**Figure 6.15** The parallelogram model for analogy problems (Rumelhart and Abrahamson, 1973): the location of $\overrightarrow{\text{vine}}$ can be found by subtracting $\overrightarrow{\text{tree}}$ from $\overrightarrow{\text{apple}}$ and adding $\overrightarrow{\text{grape}}$.

$$\hat{b}^* = \underset{x}{\text{argmax}}\ \text{distance}(x, a^* - a + b)$$

26

# Semantic properties of embeddings

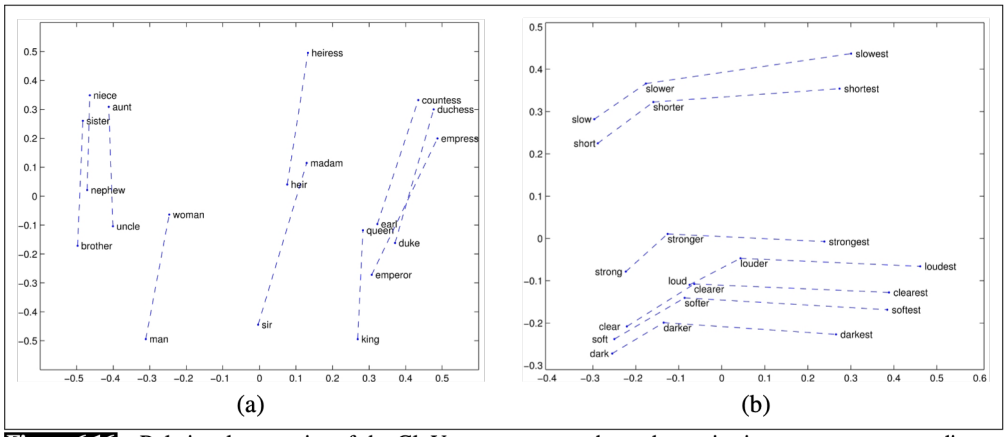



**Figure 6.16** Relational properties of the GloVe vector space, shown by projecting vectors onto two dimensions. (a) $\overrightarrow{(king)} - \overrightarrow{man} + \overrightarrow{woman}$ is close to $\overrightarrow{queen}$. (b) offsets seem to capture comparative and superlative morphology (Pennington et al., 2014).

27

# Evaluating Vector Models

- Similarity
  - Computing the correlation between an algorithm's word similarity scores and word similarity ratings assigned by humans.
  - **WordSim-353** (Finkelstein et al., 2002) is a commonly used set of ratings from 0 to 10 for 353 noun pairs; for example (*plane*, *car*) had an average score of 5.77.
  - **SimLex-999** (Hill et al., 2015) is a more difficult dataset that quantifies similarity (*cup, mug*) rather than relatedness (*cup, coffee*), and including both concrete and abstract adjective, noun and verb pairs

28

# Evaluating Vector Models

- Analogy task
  - The system has to solve problems of the form *a is to b as a* is to b**,given *a, b,* and *a** and having to find *b** (Turney and Littman, 2005)
  - covering morphology (*city:cities::child:children*)
  - lexicographic relations (*leg:table::spout::teapot*)
  - encyclopedia relations (*Beijing:China::Dublin:Ireland*)
  - **SemEval-2012 Task 2 dataset** of 79 different relations (Jurgens et al., 2012).

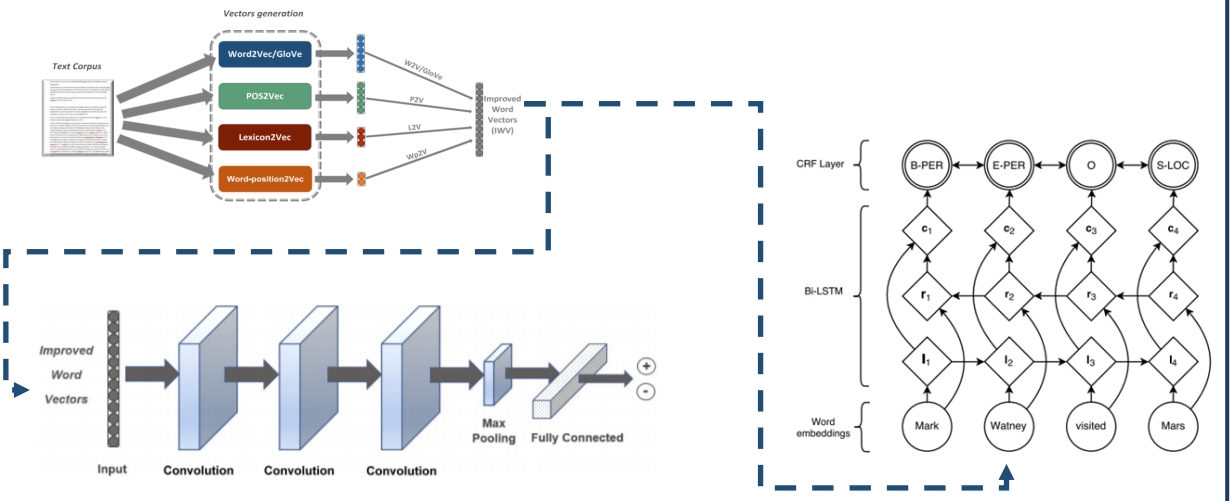
29

# Embeddings

- It is best practice to train multiple embeddings with bootstrap sampling over documents and average the results (Antoniak and Mimno, 2018).

30

# Classifiers




31

32

Questions?

33